

MACHINE LEARNING

CLASSIFICATION

NGO QUOC VIET-2026

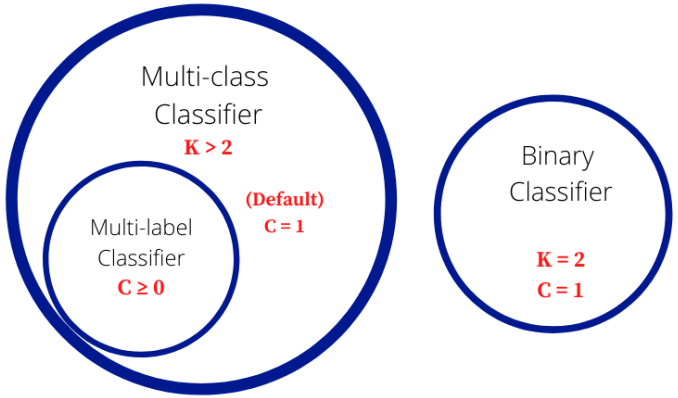
CONTENTS

- Introduction to classification problem
- Machine learning methods
  - Discriminative and Generative.
  - Parametric and Non-parametric
  - Supervised and Unsupervised
- Model Evaluation
  - Loss functions
  - Confusion Matrix
  - AUC
- Some machine learning algorithms
- Classification Simple Code

WHAT IS CLASSIFICATION PROBLEM

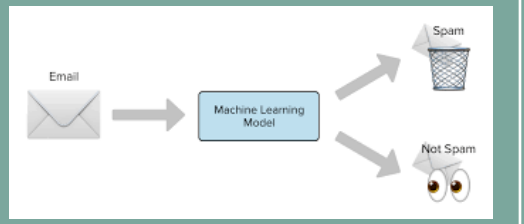
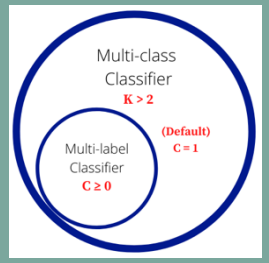
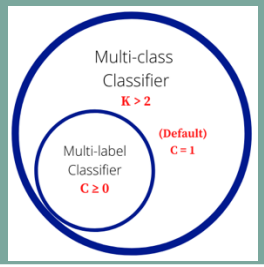
Classification refers to a predictive modeling problem where a class label is **predicted** for a given example of input data. We have 4 types:

- Binary Classification: true or not; spam or not; buy or not.
- Multi-Class Classification: classification tasks that have more than two class labels. **Classes are mutually exclusive**. **Each sample is *assigned to one and only one label.*** The number of class labels may be very large on some problems.
- Multi-Label Classification: assigns to each sample a set of target labels. **One or more class labels may be predicted for each example**.
- Imbalanced Classification: classification tasks where the number of examples in each class is unequally distributed. Ex: Medical diagnostic tests.



K = Total number of classes in the problem statement
C = Number of classes an item maybe assigned to

THE POLULAR ALGORITHMS FOR CLASSIFICATION

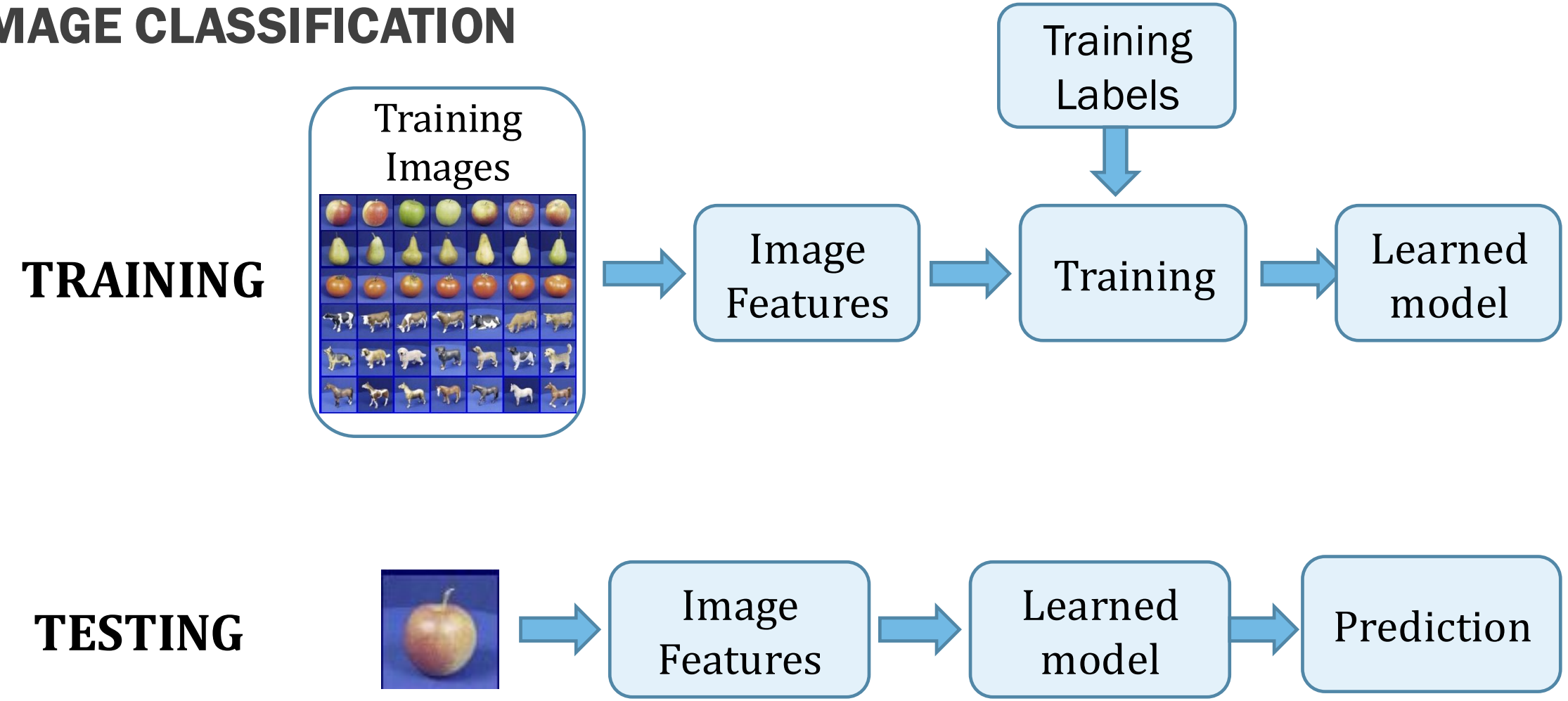
Binary	Multi Class	Multi labels	Imbalanced
			
Logistic Regression k-Nearest Neighbors Decision Trees Support Vector Machine Naïve Bayes. Ex: spam or not, disease or not, v.v.	k-Nearest Neighbors. Decision Trees. Naïve Bayes. Random Forest. Gradient Boosting. *Algorithms that are designed for binary classification can be adapted for use for multi-class.* Ex: OCR, Face classification, v.v.	Multi-label Decision Trees Multi-label Random Forests Multi-label Gradient Boosting Ex: photo classification.	*Change the composition of samples in the training dataset by undersampling the majority class or oversampling the minority class:* ***Random Undersampling ;SMOTE oversampling***. Cost-sensitive Logistic Regression. Cost-sensitive Support Vector Machines.

CLASSIFICATION

$$y = f(\boldsymbol{x})$$

- **Training:** cho tập học (*training set)* $\{(\boldsymbol{x}_1, y_1), \ldots, (\boldsymbol{x}_N, yN)\}$, ước lượng hàm dự đoán f bằng cách cực tiểu lỗi phân lớp sai trên tập học.
- **Testing:** áp dụng f vào mẫu chưa biết $\boldsymbol{x}$ và output the giá trị dự đoán $y = f(\boldsymbol{x})$.

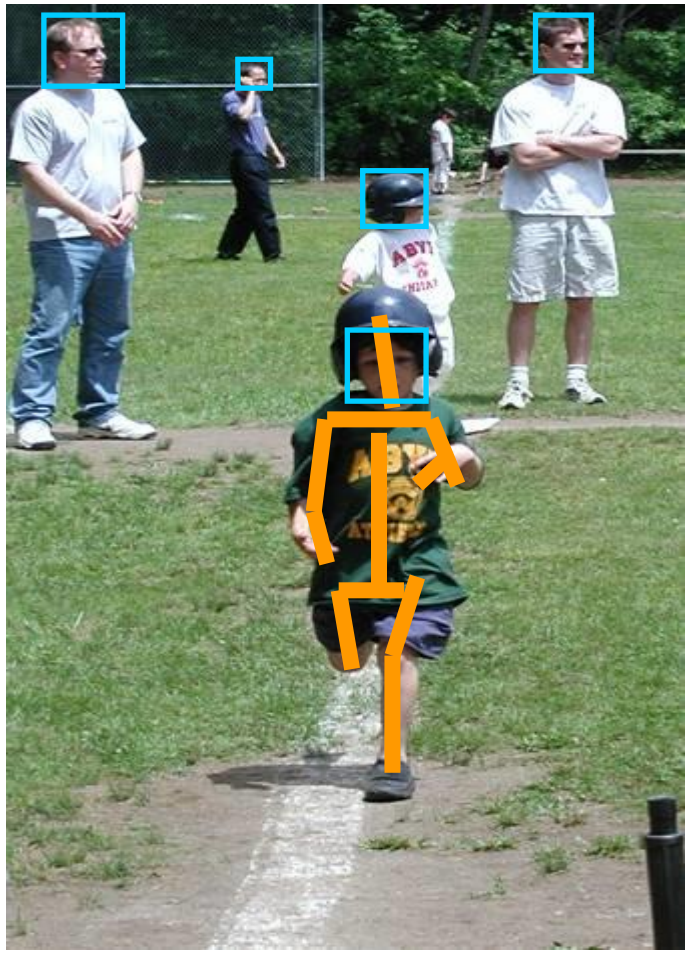
IMAGE CLASSIFICATION



DISCRIMINATIVE VS GENERATIVE MODELS

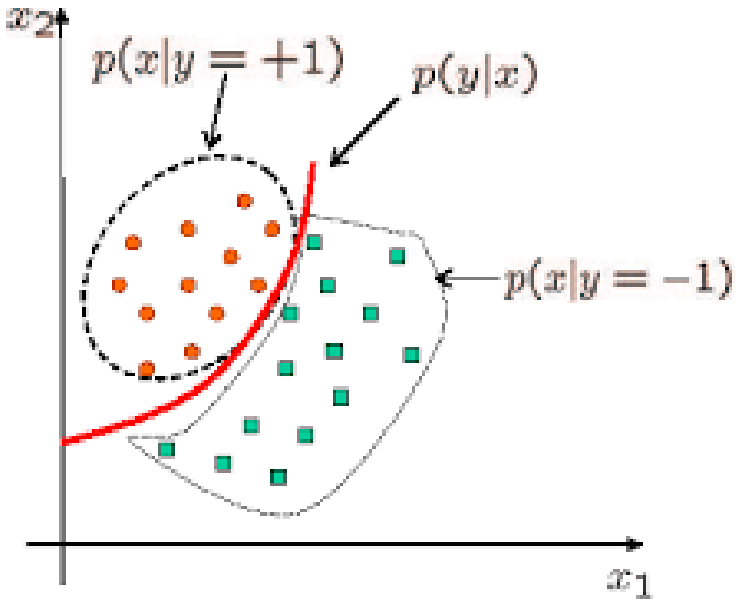
- Generative and discriminative learning are key problems in machine learning.

- If you are asking, "Are there any faces in this image?", then you would probably want to use discriminative methods.
- If you are asking, "Find a 3D model that describes the runner", then you would use generative methods.



ICCV W. Freeman and A. Blake

DISCRIMINATIVE VS GENERATIVE



$$\boldsymbol{p(y|x)}$$

Discriminative models, either explicitly or implicitly, study the posterior distribution directly.

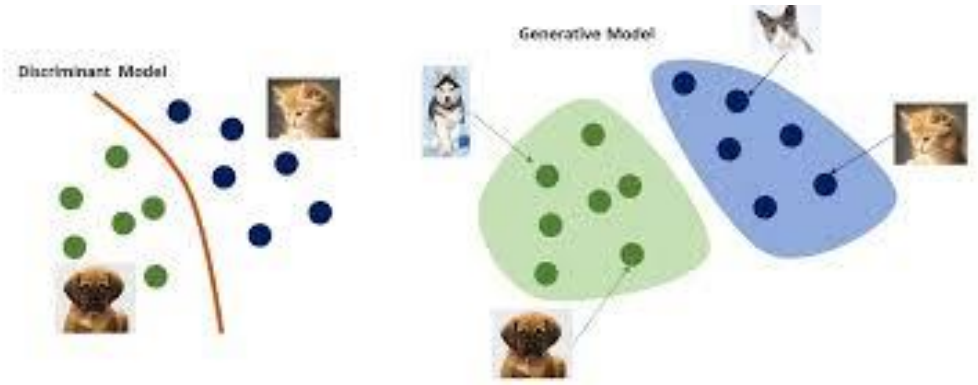
$$\boldsymbol{p(x,y);\ p(x|y), p(y)}$$

Generative approaches model the likelihood and prior separately. Estimate $p(x|y)$ to then deduce $p(y|x)$

$$p(x,y) = p(x|y), p(y);\ \ p(y|x) = \frac{p(x|y)p(y)}{\sum_y p(x|y)p(y)}$$

DISCRIMINATIVE VS GENERATIVE MODELS

- **Generative**: models where the focus is the distribution of individual classes in a dataset and the learning algorithms tend to model the underlying patterns/distribution of the data points. These models use the intuition of joint probability in theory, creating instances where a given feature (x)/input and the desired output/label (y) exist at the same time.
- **Discriminative**: also called *conditional models*, **tend to learn the boundary** between classes/labels in a dataset. The goal is to find the ***decision boundary*** separating one class from another.



DISCRIMINATIVE VS GENERATIVE

Discriminative Approaches

- Perceptron and Neural networks (Rosenblatt 1958, Windrow and Hoff 1960, Hopfiled 1982, Rumelhart and McClelland 1986, Lecun et al. 1998).
- Nearest neighborhood classifier (Hart 1968)
- Fisher linear discriminant analysis(Fisher)
- Support Vector Machine (Vapnik 1995)
- Bagging, Boosting,… (Breiman 1994, Freund and Schapire 1995, Friedman et al. 1998,)

Generative Approaches

- PCA, TCA, ICA (Karhunen and Loeve 1947, H´erault et al. 1980, Frey and Jojic 1999).
- MRFs, Particle Filtering (Ising, Geman and Geman 1994, Isard and Blake 1996)
- Maximum Entropy Model (Della Pietra et al. 1997, Zhu et al. 1997, Hinton 2002)
- Deep Nets (Hinton et al. 2006)

DISCRIMINATIVE VS GENERATIVE

- Discriminative models
  - The advantage of being more robust to outliers.
  - Do not make any assumptions about the data point.
  - Not capable of generating new data instances.
- Generative models
  - The presence of outliers affects these models to a significant extent.
  - Capable of creating new instances using probability estimates and maximum likelihood (WHY ?).

SEVERAL PAIR OF CONCEPTS

Generative

$p(x, y)$

Non-Parametric

$$y = \sum_{i=1}^{K} \alpha_i g_i(x)$$

Supervised

$\{(y_i, x_i), i = 1..N\}$

Discriminative

$p(y|x)$

Parametric

$y = f(x)$

Unsupervised

$\{(x_i), i = 1..N\}$

DISCRIMINATIVE VS GENERATIVE: EXAMPLE 1

- Problem of Cat & Dog classification with 2x2 images
- Assume that Dog follows: the upper half is always bright (pixel value ≈200), while the lower half is always dark (pixel value ≈50).
- There are training images: $X_1 = \begin{bmatrix} 200 & 200 \\ 50 & 50 \end{bmatrix}, X_2 = \begin{bmatrix} 210 & 190 \\ 40 & 60 \end{bmatrix}$
- It assumes the data follows a Normal distribution at each pixel and learns the μ, σ
- We will examine how generative models produce new data.
- Let's look at how discriminative models (e.g., Logistic Regression) learn and why they fail when asked to generate new images.

DISCRIMINATIVE VS GENERATIVE: EXAMPLE 1 (CONT.)

- What does generative model learn? The PDF P(X | Dog) is defined by parameters μ, σ → It learns and stores μ, σ.

Pixel	X_1	X_2	μ	σ
x_1	200	210	205	5
x_2	200	190	195	5
x_3	50	40	45	5
x_4	50	60	55	5

- To generate a new image, the model performs random sampling from the distribution.

- Randomly generating the 4 values of $z = [0.8, -1.2, 0.4, 0.2]$
- The model performs random sampling from the distribution $N(\mu, \sigma^2)$ for each pixel with.

$$x_i^{new} = \mu_i + \sigma_i . z_i, z_i \in [0,1]$$

  - $x_1^{new} = 205 + 5.(0.8) = 209$
  - $x_2^{new} = 195 + 5.(-1.2) = 189$
  - $x_3^{new} = 45 + 5.(0.4) = 47$
  - $x_4^{new} = 55 + 5.(0.2) = 56$
- $X_{new} = \begin{bmatrix} 209 & 189 \\ 47 & 56 \end{bmatrix}$ is a Dog with light upper half and a dark lower half

DISCRIMINATIVE VS GENERATIVE: EXAMPLE 1 (CONT.)

- Discriminative models focus solely on distinguishing between cats (Y=0) and dogs (Y = 1). It will find a boundary (hyperplane) using a formula.

$$z = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b$$

$$P(Y = 1|X) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

- Suppose that after training, the model learns the weights: $w = [-1, -1, 1, 1], b = 0$
- Assume: Image with $z > 0$, $P(\text{Dog} \mid X) > 0.5$, Conclusion: Dog.
- Request: generate new image $X = [x_1, x_2, x_3, x_4]$ from the above z and P. Discriminative model now has only one condition.

$$w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b > 0$$

$$\Leftrightarrow x_3 + x_4 > x_1 + x_2$$

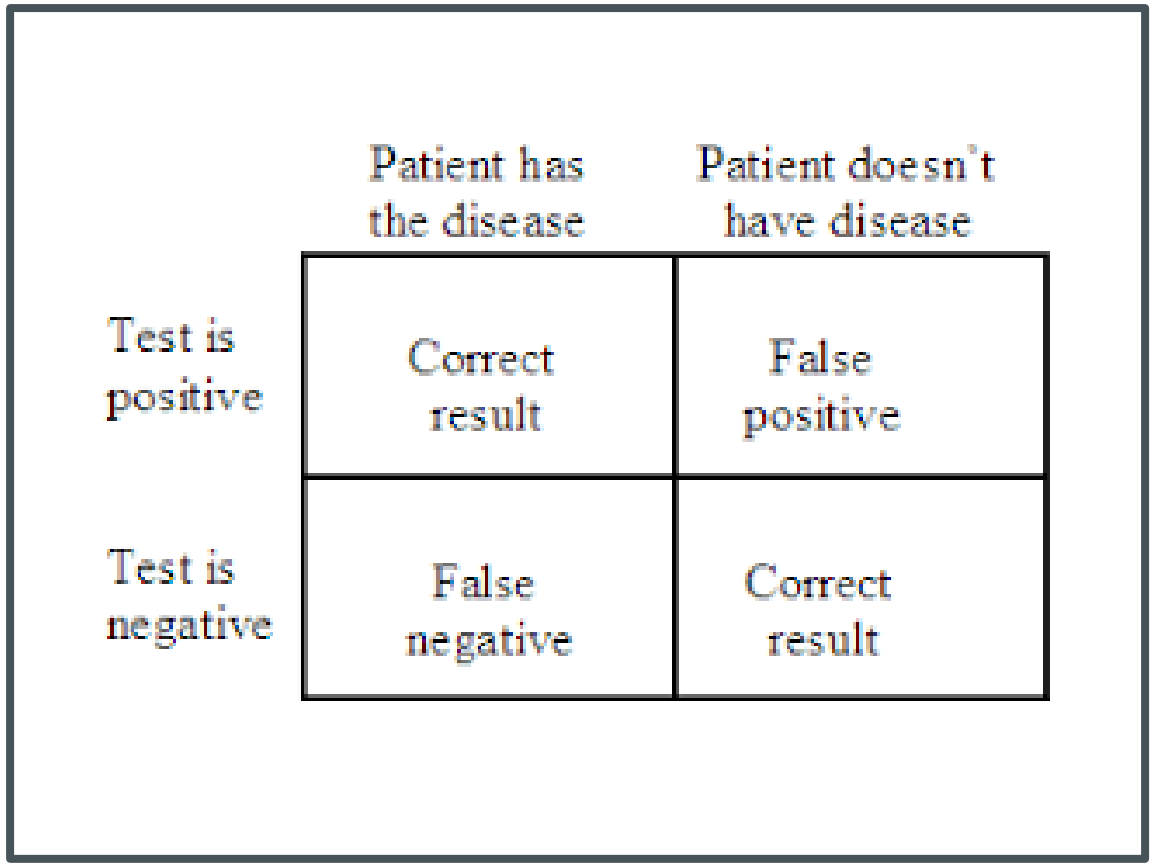
- The above inequality yields an infinite number of nonsensical solutions.
- Example: X = [0, 0, 255, 255] and [-100, -100, 0, 0] are both classified as "dog" → Incorrect.

PARAMETRIC VS NON-PARAMETRIC

Parametric	Non-parametric
Logistic regression	Nearest neighborhood
Fisher discriminant analysis	Kernel methods
Graphical models	Decision tree
Hierarchical models	Neural nets
Bagging, Boosting	Gaussian processes
…	…

Their distinction is not absolute. It roughly depends on if the number of parameters increases with the number of samples.

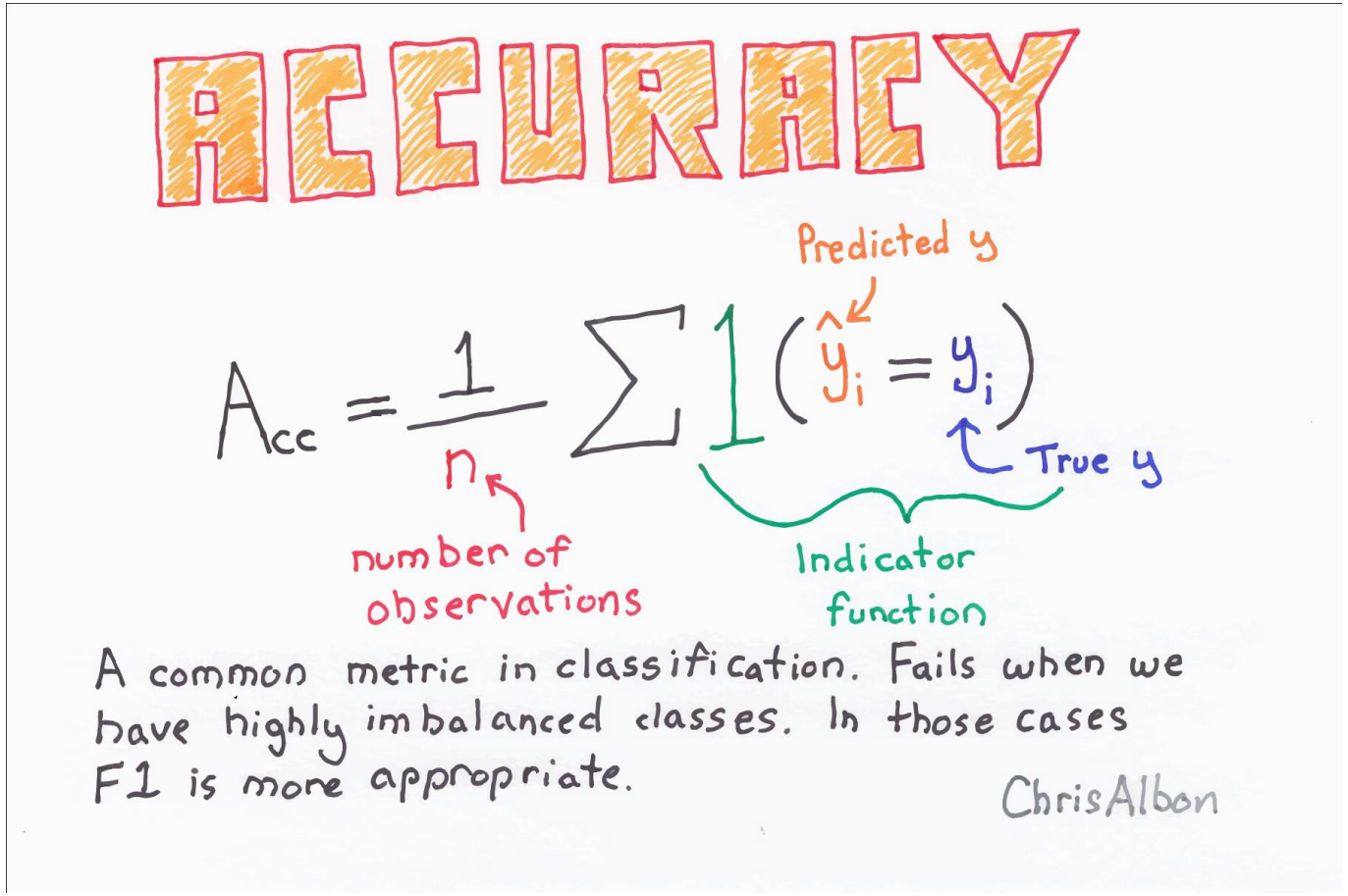
PERFORMANCE METRICS



	Patient has the disease	Patient doesn't have disease
Test is positive	Correct result	False positive
Test is negative	False negative	Correct result

- **True Positive (TP)**: the model *correctly* predicts the *positive* class.
- **True negative (TN)**: the model *correctly* predicts the *negative* class.
- **False positive (FP)**: the model *incorrectly* predicts the *positive* class.
- **False negative (FN)**: the model *incorrectly* predicts the *negative* class.

MODEL EVALUATION



https://chrisalbon.com/machine_learning/model_evaluation/accuracy/

MODEL EVALUATION

- **Accuracy**: how close a measurement is to the true **or** accepted value.

$$\boldsymbol{Accuraccy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision** (positive predict value): how close measurements of the same item are to each other. What proportion of positive identifications was actually correct

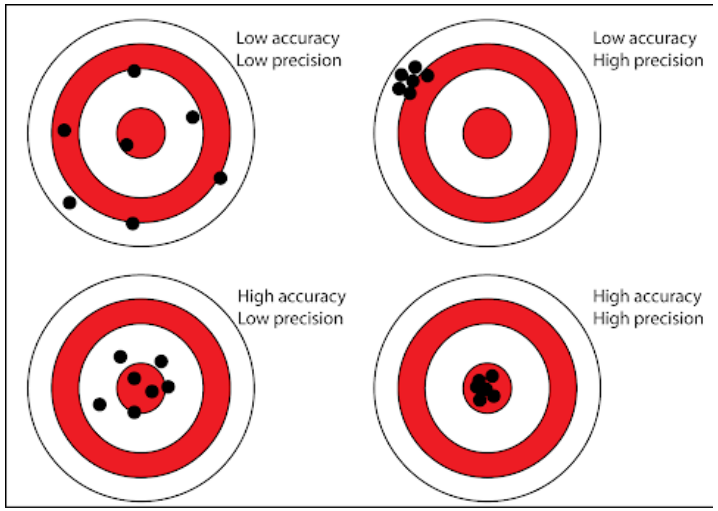
$$\textbf{Precision} = \frac{TP}{TP + FP}$$

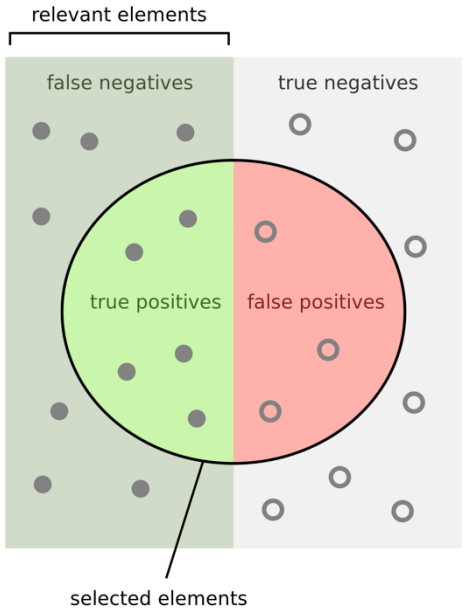
- **Recall (sensitive)**: fraction of the total amount of relevant instances that were actually retrieved

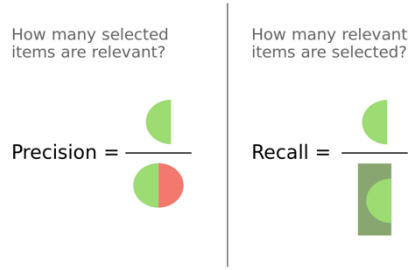
$$\boldsymbol{Recall} = \frac{TP}{TP + FN}$$

- **F1-Score**: harmonic mean of **precision and recall**

$$\boldsymbol{F_1} = \frac{2 * Precision * Recall}{Precision + Recall}$$







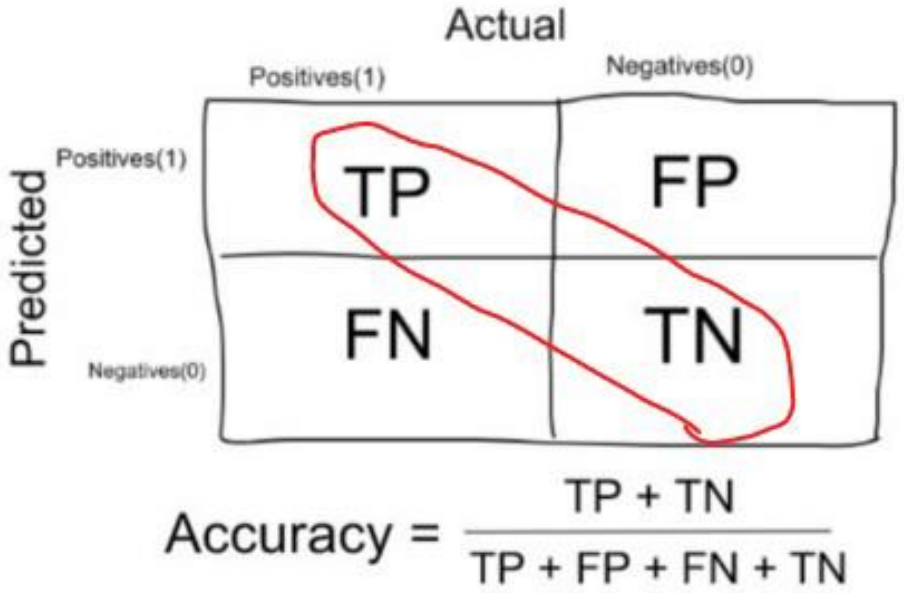
MODEL EVALUATION: EXAMPLE

- Malignant tumor prediction give the following results: 100 total;
  - Actual: 9 magniliants; 91 benign;
  - Prediction: 1 true magniliant predicted ; 1 false magniliant predicted (this guy has no magniliant); 8 false benign predicted (8 guys has magniliant); 90 true benign predicted.

True Positives: 1	False Positives: 1
False Negatives: 8	True Negatives: 90

- $Accuracy = \frac{TP+TN}{Total} = 0.91$. The classifier is doing a great job of identifying malignancies?
  - **ANSWER IS NO.** Accuracy alone doesn't tell the full story when you're working with a **class-imbalanced data set.**
- $Precision = \frac{TP}{TP+FP} = 0.5$. That means our model predicts a tumor is malignant, it is correct 50%.
- $Recall = \frac{TP}{TP+FN} = 0.11$. It correctly identifies 11% of all malignant tumors. 8 out of 9 malignancies go undiagnosed.
- $F1 = \frac{2*Precision*Recall}{Precision+Recall} = 0.15$.

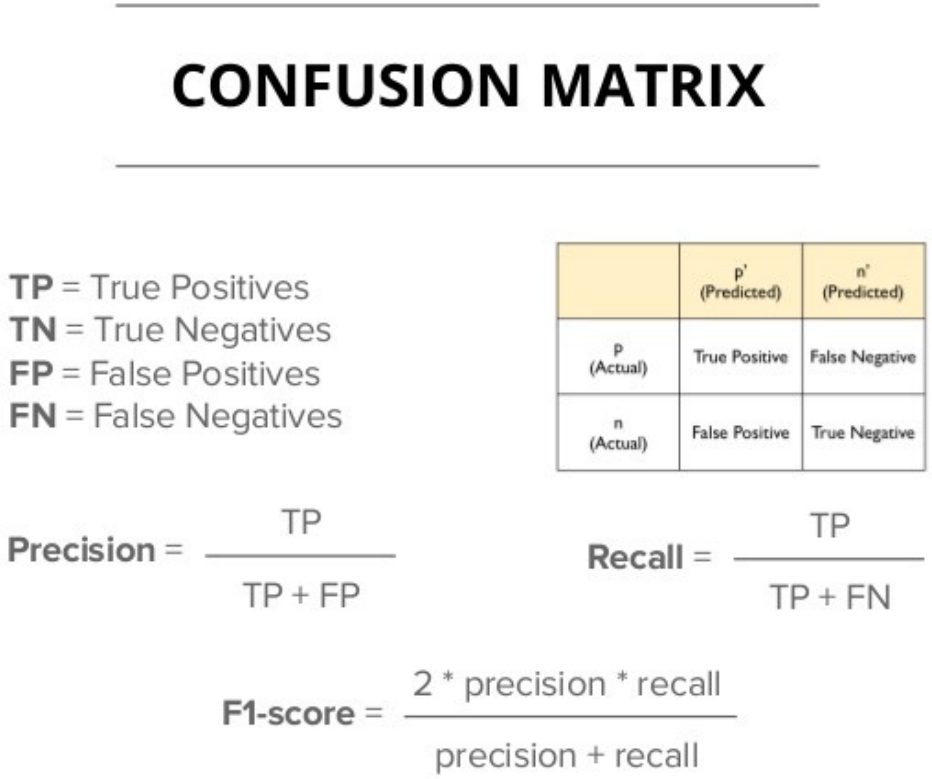
MODEL EVALUATION



Confusion matrix

Accuracy is the percentage of predictions that matches the labels.

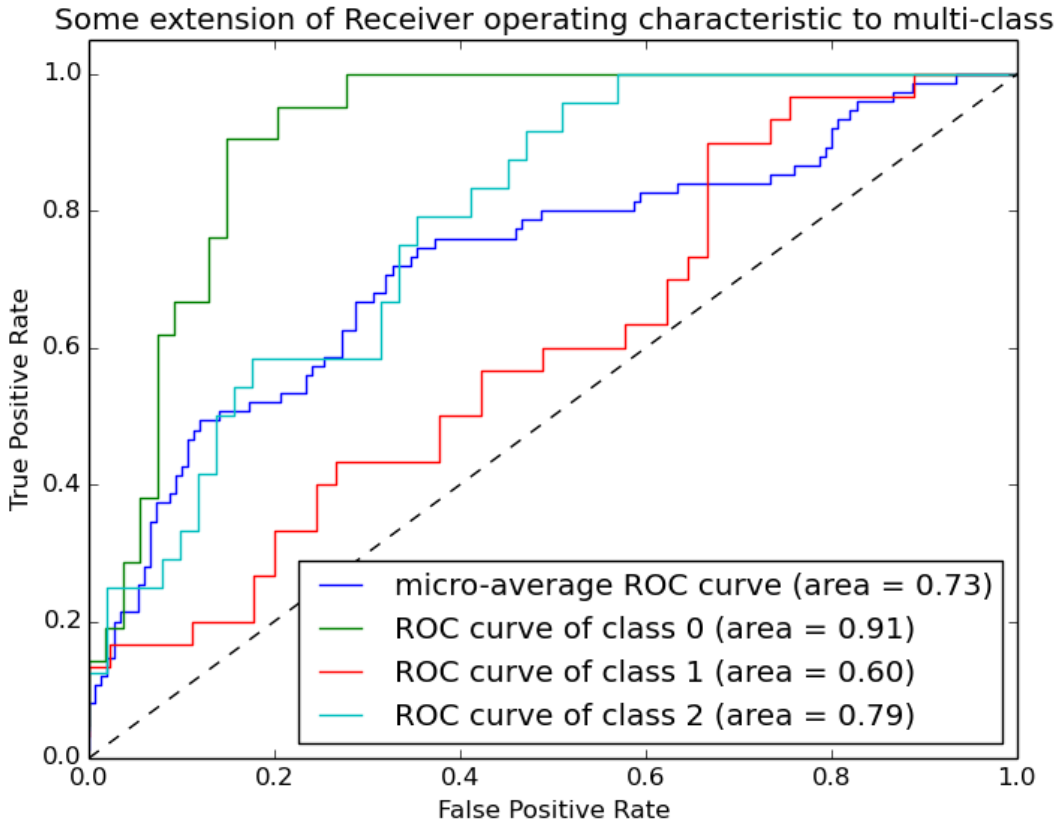
CONFUSION MATRIX



Watch: https://www.youtube.com/watch?v=j-EB6RqqjGI

AUC (AREA UNDER CURVE)

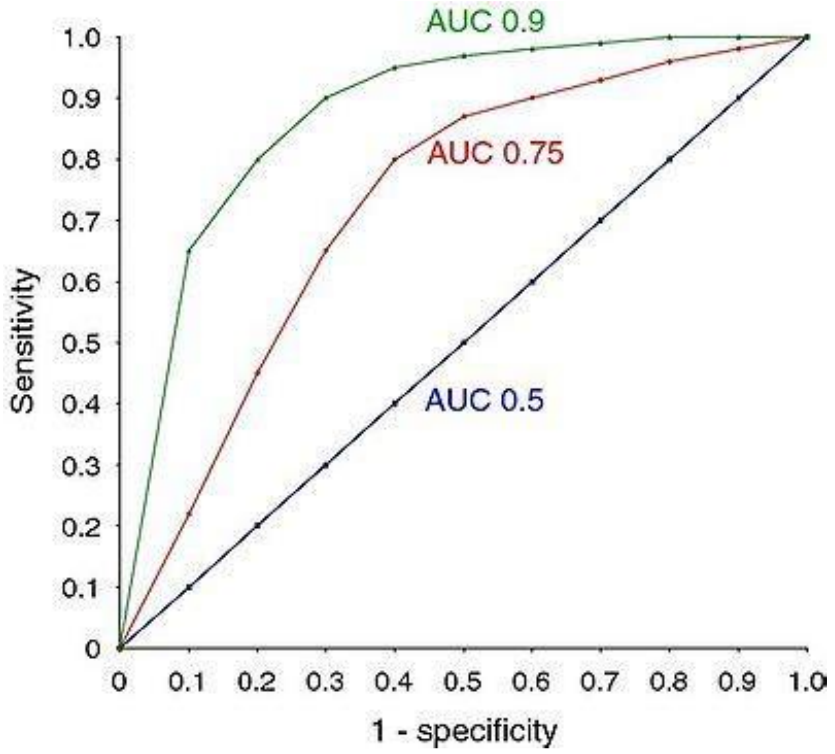
Need: AUC ~1.0



https://scikit-learn.org/0.15/auto_examples/plot_roc.html

AUC (AREA UNDER CURVE)

Need: AUC ~1.0



https://www.youtube.com/watch?v=MUCo7NvB9SI

POPULAR LOSS FUNCTIONS IN REGRESSION

Prediction output: $\hat{y}_i = f_w(x_i)$

Mean Square Error/Quadratic Loss/L2 Loss:

$$MSE = \sum_{i=1}^{n} \frac{(y_i - \hat{y}_i)^2}{n}$$

Mean Absolute Error/L1 Loss

$$MAE = \sum_{i=1}^{n} \frac{|y_i - \hat{y}_i|}{n}$$

Mean Bias Error

$$MBE = \sum_{i=1}^{n} \frac{(y_i - \hat{y}_i)}{n}$$

POPULAR LOSS FUNCTIONS IN BINARY CLASSFICATION

Common rule: penalties more for wrong prediction and vice versa.

Assume that $y_i \in \{-1, 1\}$. Prediction output: $\hat{y}_i = f_w(x_i) \in (-\infty, \infty)$, is the score representing the preferred label.

Perceptron loss: if we don't care level of correctness.

$$L_{perceptron}(y, \hat{y}) = \max(0, -y.\hat{y})$$

Hinge loss: if we consider level of correctness.

$$L_{hinge}(y, \hat{y}) = \max(0, 1 - y.\hat{y})$$

Logistic loss: distinguishes between predictions with different preferences.

$$L_{log}(y, \hat{y}) = \log_2\left(1 + exp(-y.\hat{y})\right)$$

POPULAR LOSS FUNCTIONS IN MULTI CLASSIFICATION

The predicted score of the j-th class via for the i-th data point: $s_j = f(x_i)_j$

Hinge Loss/Multi class SVM Loss: summing across all *incorrect classes* $i \neq j$, and comparing the output of our scoring function s returned for the j-th class label (incorrect class) and the $y_{i-\text{th}}$ (correct class). The score of correct category should be greater than sum of scores of all incorrect categories by some safety margin (usullaly 1).

$$L_i = \sum_{j \neq y_i} max\left(0, s_j - s_{yi} + 1\right)$$

Square Hinge Loss:

$$L_i = \sum_{j \neq y_i} max\left(0, s_j - s_{yi} + 1\right)^2$$

To derive the loss across our ***entire training set***, we simply take the mean over each individual L_i.

$$L = \frac{1}{N} \sum L_i$$

POPULAR LOSS FUNCTIONS IN MULTI CLASSIFICATION (CONT)

Hinge Loss Example





	Image #1	Image #2	Image #3
Dog	-0.39	-4.61	1.03
Cat	1.49	3.28	-2.37
Horse	4.21	1.46	-2.27

Image #1: $L_1 = max(0, 1.49 - (-0.39) + 1) + max(0, 4.21 - (-0.39) + 1) = 8.48.$ (incorrect).

Image #2: $L_2 = max(0, -4.61 - (3.28) + 1) + max(0, 1.46 - (3.28) + 1) = 0$ (correct).

Image #3: $L_3 = max(0, 1.03 - (-2.27) + 1) + max(0, -2.37 - (-2.27) + 1) = 5.2$ (incorrect).

POPULAR LOSS FUNCTIONS IN MULTI CLASSIFICATION (CONT)

Cross Entropy Loss/Negative Log Likelihood: Default setting. Cross-entropy loss increases as the predicted probability diverges from the actual label.

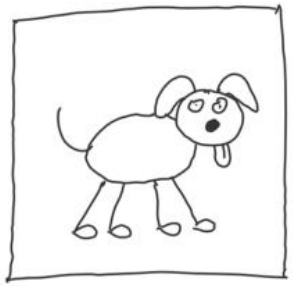
$$L_i = -\left(y_i log(\hat{y}_i)\right)$$

The Entropy loss across our ***entire sample:***

$$CE = -\sum_{i=1}^{C} y_i log(\hat{y}_i)$$

In binary classfication $$CE = -\sum_{i=1}^{2} y_i log(\hat{y}_i) = -y_1 log(\hat{y}_1) + (1 - y_1) log(1 - \hat{y}_1)$$

POPULAR LOSS FUNCTIONS IN MULTI CLASSIFICATION (CONT)



MODEL

0.5 → DOG PROBABILITY

0.3 → CAT PROBABILITY

0.2 → PANDA PROBABILITY

TARGET	PREDICTION
1	0.5
0	0.3
0	0.2

$$\text{Loss For CAT} = -p(\text{CAT}) \cdot \log q(\text{CAT})$$
$$= -0 \cdot \log q(\text{CAT})$$
$$= 0$$

$$\text{Loss For PANDA} = -p(\text{PANDA}) \cdot \log q(\text{PANDA})$$
$$= -0 \cdot \log q(\text{PANDA})$$
$$= 0$$

$$\text{Loss For DOG} = -p(\text{DOG}) \cdot \log q(\text{DOG})$$
$$= -1 \cdot \log 0.5$$
$$= 0.693\ldots$$

POPULAR LOSS FUNCTIONS IN MULTI CLASSIFICATION (CONT)

Focal Loss reduces the influence of well-predicted samples and relatively increases the weight of hard samples. It can be good for class imbalance (including the ratio of easy & hard samples).

p_t: probability of the correct class

$$p_t = \begin{cases} p, & y = 1 \\ 1 - p, & \text{otherwise} \end{cases}$$

$$FL(p_t) = -\alpha_t (1 - p_t)^{\gamma} \log(p_t)$$

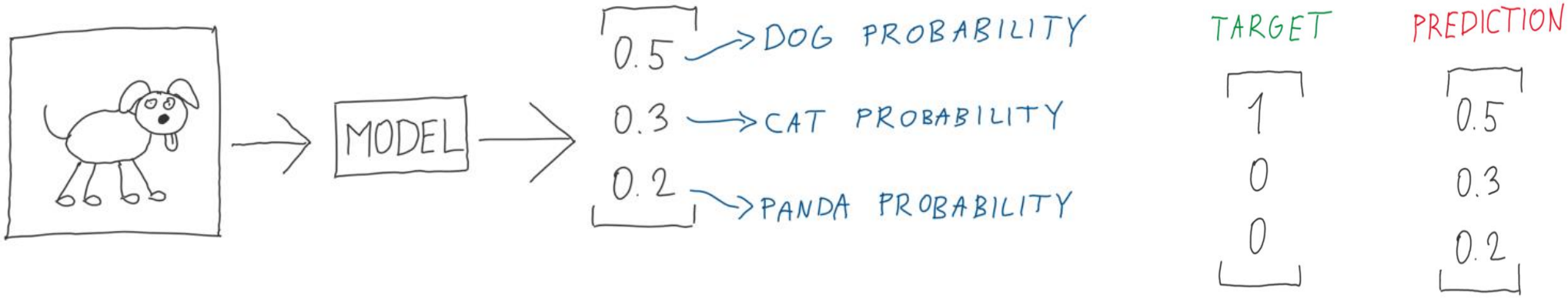
α_t: class weight. If $\alpha_t > 0.5$, the model will focus data points of true class (class 1 in binary classification).

γ: level of focus on difficult samples

High gamma, the loss will be lower for easy-to-learn data and higher for hard-to-learn data; Consequently, the model will focus on learning from the more difficult data.

When gamma is 0, Focal Loss behaves like Cross-Entropy.

FOCAL LOSS EXAMPLE



- $q_{dog} = 0.5; L_{CE} = -\log(0.5) = 0.693$
- Focal loss with $\gamma = 2$: $L_{FL} = -(1 - q_{dog})^2 = 0.173$.
  - $q_{dog} = 0.9 \rightarrow L_{FL} = 0.00105$
  - $q_{dog} = 0.1 \rightarrow L_{FL} = 1.1865$

WHY REGULARIZATION MATTERS

- The model can overlearn the training data (overfitting). Regularization is a commonly used technique in supervised learning, especially classification to avoid overfitting.
- **Add a penalty condition to the loss function**.
- For example, in logistic regression

$$loss_{total} = loss_{total} + \lambda . penalty(\theta)$$

- Regularization is a technique applied directly to the weights to control the complexity of the model.

REGULARIZATION

- Regularization is a method for “constraining” or “regularizing” the **size of the coefficients.**
- It reduces model variance and thus **minimizes overfitting.**
- If the model is too complex, it tends to reduce variance more than it increases bias, resulting in a model that is **more likely to generalize.**
- Regularization is a technique used to reduce the errors by fitting the function appropriately on the given training set and avoid overfitting.
- Logically, regularization is commonly applied to loss functions.

REGULARIZATION TECHNIQUES

- **L1 Regularisation** technique is called **LASSO (Least Absolute Shrinkage and Selection Operator).**
- **Lasso Regression** adds *"absolute value of magnitude"* of coefficient as penalty term to the loss function(L).
- **L2 regularisation** technique is called **Ridge regression.**
- **Ridge regression** adds "*squared magnitude*" of coefficient as penalty term to the loss function(L).

REGULARIZATION TECHNIQUES

- Given original loss function $f(\theta)$, λ the strength of penalty .
- L1 Regularization (Lasso): better when we want to train a sparse model, since the absolute value function is not differentiable at 0, $penalty(\theta) = \sum|\theta_i|$

$$F(\theta) = f(\theta) + \lambda\|\theta\|_1, where\ \|\theta\|_p = \sqrt[p]{\sum_{i=1}^{n}|\theta_i|^p}$$

- L2 Regularization (Ridge) for smoothing, $penalty(\theta) = \sum \theta_i^2$

$$F(\theta) = f(\theta) + \lambda\|\theta\|_2^2$$

REGULARIZATION: SIMPLE CODE

- Normalisation adjusts the data; regularisation adjusts the prediction function.

- https://medium.com/@harishreddyp98/regularization-in-python-699cfbad8622

K-NEAREST-NEIGHBORS ALGORITHM

- K nearest neighbors (KNN) is a simple algorithm that stores all available cases and classifies new cases based on a similarity measure (distance function)

- KNN has been used in statistical estimation and pattern recognition since 1970's.

K-NEAREST-NEIGHBORS ALGORITHM

- A case is classified by a majority voting of its neighbors, with the case being assigned to the class most common among its K nearest neighbors measured by a distance function.

- If K=1, then the case is simply assigned to the class of its nearest neighbor

TRAINING KNN

No training is needed or training do nothing. Sometimes, training is normalizing data.

K-NEAREST NEIGHBOR CLASSIFICATION (KNN)

- Unlike all the previous learning methods, kNN does not build model from the training data.
- To classify a test instance d, define k-neighborhood P as k nearest neighbors of d
- Count number n of training instances in P that belong to class c_j
- Estimate $\Pr(c_j|d)$ as n/k.
- **No training is needed or training do nothing. Sometimes, training is normalizing data.**
- **Classification time is linear in training set size for each test case.**

WHEN DO WE USE KNN ALGORITHM

- kNN can be used for both classification and regression. It is more widely used in classification problems in the industry.
- To evaluate any technique we generally look at 3 important aspects.

	Logistic Regression	CART	Random Forest	KNN
1. Ease to interpret output	2	3	1	3
2. Calculation time	3	2	1	3
3. Predictive Power	2	2	3	2

- kNN is commonly used for its easy of interpretation and low calculation time.

CLASSIFICATION SIMPLE CODE

- https://stackabuse.com/classification-in-python-with-scikit-learn-and-pandas/
- https://www.kaggle.com/jeffd23/scikit-learn-ml-from-start-to-finish
- https://towardsdatascience.com/solving-a-simple-classification-problem-with-python-fruits-lovers-edition-d20ab6b071d2

THANK YOU